Telecom Tower Failure Prediction MLOps Service Comprehensive Viva Voce Examination & Defense Guide

Prepared for Postgraduate Project Viva | Group 3: Nishant & Bitu | Guide: Dr. Santosh Kumar

Part 1: Quick Verbal Elevator Pitches

The 2-Minute Project Pitch (Opening Statement)

> **[2-MINUTE ELEVATOR PITCH]** "Respected examiner, our project addresses the critical problem of unscheduled cell tower outages across telecommunications networks. In mobile networks, unplanned tower downtime causes dropped calls, SLA penalties, and emergency repairs that cost 4 times more than planned maintenance.
>
> We engineered an end-to-end MLOps predictive maintenance system. Using 1,500 historical tower sensor records with an authentic 20.8% failure incidence, our system engineers 5 physical domain indicators—such as thermal stress ratio and 48V battery deviation. We trained a cost-sensitive Random Forest classifier achieving 82% accuracy and 0.8052 ROC-AUC on a held-out test split, outperforming a linear baseline.
>
> Critically, we did not leave the model in a notebook. We deployed it as a containerized FastAPI service with Pydantic validation, single/batch prediction, Prometheus telemetry, and an automated Bash watchdog daemon that detects outages and self-heals in 6 seconds. Our test suite includes 69 automated tests and verified 100% parity across 50 sample rows between offline and online scoring."

The 5-Minute Technical Defense Walkthrough

When the examiner requests a deep walkthrough of the architecture:

- **1. Ingestion & Transformation:** Raw sensor inputs (temperature, voltage, power, fan RPM, humidity, traffic, age) enter `TowerFeatureTransformer`. It clips physical sensor bounds, imputes missing values using medians learned exclusively during training, and computes domain ratios without data leakage.
- **2. Model Layer:** Trained with 100 estimators, depth 8, and balanced class weights to penalize minority false negatives. Evaluated with Confusion Matrices, ROC-AUC (0.8052), and PR-AUC (0.5880). Cross-validation ROC-AUC achieved 86.09%, explaining a prior metric ambiguity.
- **3. Serving Layer:** Built on FastAPI. The model loads exactly once into RAM at startup via ASGI lifespan. Inputs are strictly validated: impossible readings return HTTP 422, while an empty payload list returns 422, guaranteeing zero server crashes.
- **4. Observability Stack:** Docker multi-stage container running as non-root `appuser`. Managed via Docker Compose alongside Prometheus and Grafana. Exposes request counters, latency histograms (p50/p95/p99), and failure ratios.
- **5. Reliability & Self-Healing:** Cron-scheduled Bash watchdog queries `/health` every minute. If degraded, it restarts the container, verifies recovery, and manages log rotation via logrotate.

Part 2: 45 Substantive Technical Examination Questions & Answers

Q1: Why is failure prediction in mobile towers framed as a 48-hour window?

A 48-hour forward horizon matches field maintenance logistics. Dispatching a certified technician with replacement hardware (rectifiers, backup batteries, or cooling blowers) to remote or rural towers typically requires 12 to 24 hours. A 1-hour horizon is too short for dispatch, while a 30-day horizon suffers high variance and changing weather conditions.

Q2: What is the primary business metric for predictive maintenance in telecom?

The primary business metric is Mean Time Between Failures (MTBF) and reduction in unplanned downtime. Economically, preventive field replacement costs ~1/4 of emergency outage repair, which incurs overtime labor, emergency generator fuel, and telecom regulatory SLA penalties.

Q3: Why is accuracy a deceptive metric for this dataset?

With a 20.8% failure incidence, a naive dummy model predicting 'Normal (0)' for every single tower achieves 79.2% accuracy while detecting exactly 0% of failures. In reliability engineering, Recall (sensitivity) and PR-AUC are far more meaningful than accuracy alone.

Q4: What is the difference between planned maintenance and predictive maintenance?

Planned maintenance replaces parts on a fixed calendar schedule regardless of wear, wasting component lifespan. Predictive maintenance uses real-time telemetry to predict when degradation crosses failure thresholds, optimizing maintenance cycles.

Q5: What are the main physical failure modes of a cell tower?

The top failure modes are: (1) thermal runaway from cooling fan failure; (2) rectifier power supply breakdown; (3) backup battery bank voltage collapse under grid outage; and (4) RF power amplifier burnout under high traffic.

Q6: Describe your dataset source, dimensions, and target distribution.

The dataset `telecom_tower_failure.csv` comprises 1,500 rows and 10 columns: 8 numerical sensor channels, 1 identifier (`Tower_ID`), and 1 binary target (`Failure_Within_48Hrs`). There are 1,188 Normal towers (79.2%) and 312 Failed towers (20.8%).

Q7: How did you handle missing values in the sensor telemetry?

A total of 180 missing values were observed across the 8 sensor columns. We handle missing values via median imputation learned strictly on `X_train` within `TowerFeatureTransformer`. Live inference uses these learned medians rather than batch statistics.

Q8: Why is Tower_ID excluded from feature transformation?

`Tower_ID` is an alphanumeric administrative identifier (e.g., TWR-09001). Using it as a feature would cause extreme overfitting and data leakage, as the model would memorize individual towers rather than learning sensor degradation physics.

Q9: What physical sensor limits did you establish for data bounds?

Based on cell tower hardware specs: Temperature (-20 to 85°C), Battery Voltage (20 to 65V), Power (50 to 3000W), Signal (0 to 100%), Fan Speed (500 to 6000 RPM), Humidity (0 to 100%), Traffic (0 to 2000), and Age (0 to 30 years).

Q10: What did the correlation analysis reveal about the target?

The Pearson correlation heatmap revealed that Temperature_C and Thermal_Stress_Ratio had the strongest positive correlations with failure, while Fan_Speed_RPM had a negative correlation.

Q11: Why did you create the Thermal_Stress_Ratio feature?

High temperature alone may not indicate failure if cooling fans are running at full 6000 RPM. However, high temperature coupled with low fan RPM indicates cooling system failure. The ratio `Temperature_C / (Fan_Speed / 1000)` captures this interaction directly.

Q12: How does Power_Per_Traffic handle idle towers with zero traffic?

Towers with zero active calls would cause a division-by-zero error. We formulate the feature as `Power_Consumption_W / (Traffic_Load + 1.0)`, providing mathematical stability across all idle and active operating states.

Q13: What is the physical significance of Battery_Voltage_Dev48?

Cellular base station DC power plants operate on a nominal -48V DC bus standard. Deviations `|Battery_Voltage - 48.0|` indicate either severe battery discharge (< 42V) or over-voltage rectifier failure (> 54V).

Q14: Explain the Heat_Humidity_Index.

`Temperature_C * (Humidity_Percent / 100.0)`. High heat combined with high relative humidity accelerates electrochemical migration and corrosion on printed circuit boards inside the equipment cabinet.

Q15: How did you ensure zero data leakage in your preprocessing pipeline?

The dataset was split into 80% train and 20% test before fitting. All imputation medians were calculated strictly on `X_train`. The transformer is pickled inside the model bundle, guaranteeing that serving uses training medians.

Q16: Why did you select Random Forest over deep learning or simple linear regression?

Tabular sensor datasets under 50,000 rows are dominated by tree ensembles (Grinsztajn et al., NeurIPS 2022). Random Forest handles non-linear physical interactions, requires no scaling, is robust to outliers, and does not overfit easily.

Q17: What hyperparameters were chosen for the Random Forest and why?

`n_estimators=100` provides stable probability estimates; `max_depth=8` prevents leaf memorization; `min_samples_leaf=4` ensures leaf regularization; and `class_weight='balanced'` penalizes minority failure errors.

Q18: What was your model's exact test set performance?

On the held-out 300-sample test set: Accuracy = 82.00%, Recall = 54.84%, Precision = 56.67%, Specificity = 89.08%, F1-Score = 0.5574, ROC-AUC = 0.8052, and PR-AUC = 0.5880.

Q19: How did the baseline Logistic Regression compare?

Logistic Regression achieved higher recall (67.74%) but at the cost of poor precision (46.15%), lower accuracy (77.00%), and 49 false alarms (compared to only 26 for Random Forest).

Q20: What was the confusion matrix for the Random Forest on test data?

TN = 212 (healthy towers correctly cleared), FP = 26 (unnecessary inspections), FN = 28 (missed failures), and TP = 34 (prevented outages). Total = 300.

Q21: Why did earlier documentation claim 85% recall, whereas your evaluation shows 54.84%?

Our audit investigated the raw training logs and metadata. The 5-fold cross-validation ROC-AUC was 86.08% (`cv_roc_auc_mean: 0.86086`). In prior informal discussions, this 86% CV ROC-AUC was mistakenly quoted as '85% recall'. Test recall at threshold 0.50 is 54.84%.

Q22: Can your model achieve 85% recall, and what would be the trade-off?

Yes. By tuning the classification threshold down from 0.50 to approximately 0.28, recall rises to 85%. However, this increases false positive alarms from 26 to over 65, diverting field maintenance teams from routine duties.

Q23: What is the optimal operational threshold for telecom dispatch?

If the cost of an outage is $5,000 and the cost of an inspection is $500 (10:1 ratio), the optimal Bayesian decision threshold is ~0.35, which yields 77.4% recall while maintaining acceptable precision.

Q24: What is the difference between ROC-AUC and PR-AUC?

ROC-AUC plots True Positive Rate against False Positive Rate. When the negative class is dominant, ROC-AUC can remain high even if false positives increase. PR-AUC plots Precision vs Recall and is much more informative for imbalanced problems.

Q25: What was the top feature by Gini importance in your Random Forest?

`Battery_Voltage_Dev48` and `Thermal_Stress_Ratio` were the top two features, confirming that voltage instability and thermal cooling breakdown are the primary physical predictors of failure.

Q26: Why did you choose FastAPI over Flask or Django?

FastAPI provides native asynchronous ASGI performance, automatic OpenAPI documentation, tight integration with Pydantic v2 for runtime type safety, and native dependency injection.

Q27: How does your service ensure the model is loaded only once?

We use FastAPI's ASGI `lifespan` context manager. On startup, `joblib.load()` loads the model bundle into a global memory dictionary `MODEL_STATE`. Subsequent inference requests query this in-memory object in microseconds.

Q28: How does your API handle invalid sensor inputs?

Inputs outside sensor boundaries (e.g., Temp = 250°C) or non-numeric strings are intercepted by Pydantic validators and rejected with HTTP 422 Unprocessable Content. The server never crashes.

Q29: How did you fix the empty list HTTP 500 bug?

Sending `[]` to `/predict` previously caused scikit-learn to throw `ValueError` on 0-sample arrays. We added validation checking `if len(payload) == 0:` and returning a structured HTTP 422 error response.

Q30: How did you prove prediction parity between offline and online scoring?

In `test_pytest_suite.py`, 50 rows from `load_test_requests.csv` were scored offline via joblib and simultaneously sent to the `/predict/batch` endpoint. We asserted that all 50 predictions and probabilities matched 100%.

Q31: What are the key stages in your multi-stage Dockerfile?

Stage 1 (`builder`) installs build dependencies and compiles wheels. Stage 2 (`runtime`) uses `python:3.11-slim`, copies pre-built wheels, creates a non-root `appuser`, and sets up the Uvicorn entrypoint.

Q32: Why is running containers as a non-root user important?

Running as root in a container presents container breakout vulnerabilities. If an attacker exploits an application vulnerability, non-root execution restricts their access to host system resources.

Q33: What restart policy is configured in docker-compose.yml?

`restart: unless-stopped`. If the container crashes or the server reboots, Docker daemon automatically restarts the container unless an administrator explicitly stopped it.

Q34: How is Prometheus configured to scrape your service?

`prometheus.yml` defines a scrape job targeting `telecom-tower-service:8000` with a scrape interval of 5 seconds on the path `/metrics`.

Q35: What is the purpose of the health check in Docker?

The `HEALTHCHECK` directive runs `curl -f http://localhost:8000/health || exit 1` every 30 seconds. This allows Docker Compose and orchestrators to detect deadlocks and restart unhealthy containers.

Q36: Explain the difference between Counter, Gauge, and Histogram in Prometheus.

A Counter only increases (e.g., `http_requests_total`). A Gauge can go up and down (e.g., `tower_failure_prediction_ratio`). A Histogram samples observations into configurable buckets to calculate percentiles (e.g., `http_request_duration_seconds`).

Q37: What alert conditions did you configure and defend?

Alert 1: High Latency (p95 > 300ms for 2m), indicating thread pool exhaustion. Alert 2: High Failure Ratio (failure prediction share > 40%), indicating model drift or widespread severe weather damage.

Q38: Why was a latency threshold of 300ms chosen instead of 100ms or 800ms?

Load testing showed normal p95 latency under 100 concurrent users is ~92ms. A 100ms threshold causes false alarms during transient spikes. An 800ms threshold triggers too late, after users experience timeouts.

Q39: How can an on-call engineer detect model drift using your dashboard?

The `tower_failure_prediction_ratio` gauge tracks the rolling fraction of failure predictions. If this departs significantly from the baseline 20.8% (e.g., dropping to 5% or spiking to 60%), it signals input sensor drift.

Q40: What were your load test results under 10, 100, and 500 concurrent users?

10 users: 85.4 RPS, p95 = 24ms, 0% errors. 100 users: 214.2 RPS, p95 = 92ms, 0% errors. 500 users: 312.0 RPS, p95 = 680ms, 1.8% timeouts. Breaking point was identified at ~310 RPS.

Q41: What bottleneck improvement did you implement?

We increased Uvicorn workers from 1 to 4 and optimized DataFrame construction in `_score_records`. This reduced p95 latency at 100 users from 92ms to 28ms and raised maximum throughput to > 450 RPS.

Q42: What capacity configuration do you recommend for production?

A 4-worker Uvicorn deployment provisioned with 2 vCPUs and 2 GB RAM. This comfortably accommodates a peak sustained load of 250 RPS with p95 latency under 50ms.

Q43: How does your automated Bash watchdog script work?

`healthcheck_watchdog.sh` executes via cron every minute. It curls `/health`. If non-200 or timeout occurs, it logs the incident, issues `docker restart`, waits 5 seconds, verifies health, and writes the recovery timestamp.

Q44: How does logrotate prevent server disk exhaustion?

`telecom_logrotate.conf` rotates watchdog and service logs weekly, compresses old logs with gzip, and deletes logs older than 4 rotation cycles, ensuring logs never consume more than 50 MB.

Q45: If your service crashes right now during this viva, how will it recover?

Docker's `restart: unless-stopped` policy restarts the container immediately. Furthermore, our cron watchdog routine detects the outage on its next probe and issues a restart command if Docker daemon hangs.

Part 3: 10 Examiner Trap Questions & Defenses

Trap 1: "Why is your recall only 54.84%? Isn't that too low for production?"

Defense: 'Sir, 54.84% is evaluated at the default balanced threshold of 0.50 on un-resampled test data. In telecom maintenance, threshold selection depends on the cost ratio between false alarms ($500) and missed catastrophic failures ($5,000). By lowering the threshold to 0.35, our model achieves 77.4% recall. We reported 54.84% honestly rather than fabricating an artificial 90% score.'

Trap 2: "Did you balance your test set using SMOTE?"

Defense: 'No, sir! Applying SMOTE to the test set is a severe data leakage violation. SMOTE or class re-weighting must only ever be applied to the training set. The test set must preserve the natural empirical distribution (20.8% failure incidence) to reflect real production performance.'

Trap 3: "Your API returns version as RandomForestClassifier. What if I want to deploy XGBoost next week?"

Defense: 'Our architecture is completely decoupled. The API loads a pickled bundle containing both the model and the transformer. As long as the candidate model implements Scikit-learn's `predict_proba()` API and accepts the 13 transformed features, it can be dropped into `artifacts/` with a version bump without changing a single line of API code.'

Trap 4: "What happens if a tower sensor sends a temperature of 200°C?"

Defense: 'Our Pydantic schema in `schemas.py` defines `Temperature_C` with a physical constraint `le=85.0`. The request is intercepted at the API gateway and rejected with HTTP 422 Unprocessable Content. It never reaches the model and cannot cause an internal server error.'

Trap 5: "Why not use Kubernetes instead of Docker Compose?"

Defense: 'Kubernetes introduces significant operational overhead and resource consumption that is unjustified for a single-server deployment. For our requirements (Task 4), Docker Compose provides single-command startup, native service discovery, and automated restart policies with minimal resource overhead.'